PROJECT OVERVIEW · EASY-LANGUAGE GUIDE

# Session-Based Recommendation System Using Liquid Neural Networks

A student project that predicts the next item a visitor may interact with, based on items already seen in the same shopping session.

## The idea

If a session looks like **Laptop** → **Mouse** → **Keyboard**, the system ranks likely next items. It compares three approaches: a simple popularity baseline, a GRU sequence model, and an LTC (Liquid Time-Constant) sequence model.

**Popularity**

Which items occur most in training?

**GRU**

What sequence patterns did the model learn?

**LTC**

How can order plus elapsed time help?

## Project flow

**1** RetailRocket events → **2** Build sessions and splits → **3** Train three models → **4** Measure ranking quality → **5** Recommend top five

**In short:** the input is a recent sequence of item IDs; the output is a ranked list of items from the training catalogue. The IDs are anonymous and this project does not identify individual people.

Project source and fuller README: github.com/dhrumi2004/session-recommendation-lnn

01 · DATA AND PREPARATION

# What data is used?

The project uses the public RetailRocket e-commerce dataset. It focuses on `events.csv`, which records an anonymous visitor, item, event type, and timestamp.

| | | |
|---|---|---|
| **2,756,101**<br>event rows read; no invalid rows removed. | **30 minutes**<br>inactivity gap used to start a new session. | **41,261**<br>items in the training-only recommendation catalogue. |

## How raw clicks become training examples

1. Check required columns and discard missing identifiers or invalid timestamps.
2. Sort each visitor's events by time and make a new session after a gap longer than 30 minutes.
3. Drop sessions shorter than two interactions.
4. Split complete sessions chronologically into train, validation, and test. Sessions crossing a split boundary are discarded.
5. Build the item vocabulary from training sessions only. ID 0 is reserved for padding.
6. Turn each session into prefix → next-item examples.

```
[A, B, C, D]
[A]       → B
[A, B]    → C
[A, B, C] → D
```

## Processed data summary

- 382,780 eligible sessions were constructed; 25 crossing a time boundary were discarded.
- 685,268 training examples, 66,408 validation examples, and 61,389 test examples were generated.
- 22,105 validation and 27,137 test target occurrences were outside the training vocabulary. These cannot be ranked and are counted separately.

> **Leakage prevention:** the vocabulary and item-frequency threshold come from training data only. LTC elapsed-time inputs use only events already in the history—never the target's timestamp or future time gap.

Dataset listing: Kaggle · RetailRocket Ecommerce Dataset. Listed license: CC BY-NC-SA 4.0; follow its terms before sharing the data.

Dataset file used locally: data/raw/archive/events.csv · Dataset itself is not in the GitHub repository

02 · THE THREE MODELS

# How do the recommenders work?

### Popularity baseline

Counts each item's appearances in the training sessions and recommends the most frequent ones first. It is simple and does not use the current session order. This is a useful reference point.

### GRU recommender

Maps item IDs to learned numerical representations called embeddings. A GRU reads them in order and builds a summary of the session. A final scoring layer compares that summary with every item in the training catalogue.

### LTC recommender — main research model

Also reads embedded items in order, but its recurrent state follows Liquid Time-Constant dynamics. It receives elapsed seconds between events already seen in the session. The ncps LTC cell updates the hidden state, and a final layer scores all candidate items.

## Training in plain words

- The model guesses the next item and compares its scores with the correct item using cross-entropy loss.
- Adam updates the model weights over mini-batches.
- Validation loss is checked each epoch; the best validation checkpoint is saved, with early stopping available.
- Padding is only for batching sequences of different lengths. It is excluded from item recommendations.

**Experiment settings:** GRU and LTC each ran for three epochs with 32-dimensional embeddings, hidden size 32, batch size 512, learning rate 0.001, seed 42, and a maximum history of 10 items. Training and evaluation used Apple MPS on the development Mac.

MPS is Apple's GPU compute backend. CPU can also be selected; CUDA is for compatible NVIDIA GPUs.

Implementation: PyTorch · LTC cell: ncps · Saved weights: artifacts/checkpoints/

03 · RESULTS

# How well did the models do?

All models were evaluated on the same 61,389 mapped test examples, with the same 41,261 candidate items and the most recent 10 history items.

| Model | Recall@5 | Recall@10 | MRR@5 | MRR@10 |
|---|---|---|---|---|
| Popularity | 0.0064 | 0.0114 | 0.0044 | 0.0051 |
| GRU | 0.1886 | 0.2155 | 0.1455 | 0.1491 |
| **LTC** | **0.2432** | **0.2680** | **0.1993** | **0.2027** |

| Recall@K | MRR@K |
|---|---|
| How often the correct next item appears among the first K recommendations. | Rewards a correct item more when it is near the top. Rank 1 scores 1; rank 2 scores 1/2; a miss within K scores 0. |

## What can we conclude?

For this particular RetailRocket split and training setup, LTC scored higher than GRU and popularity on all four metrics. This result supports further study of LTC for this task, but it does **not** prove LTC is always superior.

## Important limitations

- 27,137 test target occurrences were not in the training catalogue and were excluded from the metric denominator.
- Only the last 10 history items were used, even when a session had more interactions.
- All event types are treated as interactions; views, carts, and purchases are not weighted differently.
- The results come from one chronological split and a small fixed training setup—not a broad benchmark or live-store trial.
- Recommendation scores are model scores, not guaranteed purchase probabilities.

Full values and evaluation settings: artifacts/figures/test_metrics.json

# Where is everything?

**src/data.py**

Validates events, creates sessions, splits, and examples.

**src/train.py**

Trains GRU or LTC and saves the best checkpoint.

**src/evaluate.py**

Computes Recall/MRR, results JSON, and charts.

**src/recommend.py**

Loads saved artifacts and recommends top-five items.

**src/models/**

Popularity, GRU, and LTC model implementations.

**tests/**

27 automated tests for data, models, metrics, and inference.

## Reproduce the workflow (run from project root)

```
python3 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt

.venv/bin/python -m src.data \
  --input data/raw/archive/events.csv --output data/processed

.venv/bin/python -m src.train --model gru --epochs 3 \
  --batch-size 512 --embedding-dim 32 --hidden-size 32 \
  --max-sequence-length 10 --seed 42 --device auto

.venv/bin/python -m src.train --model ltc --epochs 3 \
  --batch-size 512 --embedding-dim 32 --hidden-size 32 \
  --max-sequence-length 10 --seed 42 --device auto

.venv/bin/python -m src.evaluate \
  --device auto --batch-size 512 --max-sequence-length 10

.venv/bin/python -m unittest discover -s tests -v
```

## Try the recommendation command

```
.venv/bin/python -m src.recommend \
  --model ltc --items 461686,257040,309778
```

Optional --timestamps-ms values (Unix milliseconds, one per item) let LTC use elapsed times observed in the history. Unknown IDs are reported and skipped. Use --model gru or --model popularity to compare other recommendations.

**To recreate the experiment:** download RetailRocket from Kaggle and put the extracted events.csv in data/raw/archive/. Data, processed files, checkpoints, and charts are intentionally not tracked in Git because they are large and can be regenerated.

Public source repository: github.com/dhrumi2004/session-recommendation-lnn · README.md has fuller notes